

Use Case Descriptions

Use Case: Start Game Search

Iteration: 1

Primary Actor: Player

Goal in Context: Develop a feature for the user to join the matchmaking queue to join a match.

Preconditions: User has a player account and is authorized to play the game

Trigger: Player clicks the start game button

Scenario:

1. Player wants to play any one of the three games
2. Player clicks the start game button
3. The matchmaking system connects player to matchmaking system

Post-Condition: Player has entered matchmaking queue

Exceptions:

1. Player is blacklisted and cannot enter matchmaking queue.
2. Matchmaking queue is not entered due to network connection error.

Priority: High, required to play games

When Available: First development iteration

Frequency of Use: High

Channel to actor: Player UI that has the button

Secondary Actor: Player UI trigger

Channel to Secondary Actor: Game Screen

Open Issues:

1. Player cannot use screen until the matchmaking queue is not entered.

Use Case: Skilled Based Matchmaking

Iteration: 1

Primary Actor: System

Goal in Context: Match players based on their skill level for a game

Preconditions: Player is in the matchmaking queue

Trigger: Player enters matchmaking queue

Scenario:

1. Player has entered matchmaking queue
2. Matchmaking system finds similar skilled players in a game.
3. Matchmaking system signals players to be grouped.

Post-Condition: Player are grouped together

Exceptions:

1. Matchmaking groups different skilled players due to lack of users playing the game.
2. Network interrupt encountered.

Priority: High, required to play games

When Available: First development iteration

Frequency of Use: High

Channel to actor: Game Screen

Secondary Actor: Database

Channel to Secondary Actor: Game Screen

Open Issues:

1. What happens if one of the players decide to cancel matchmaking midway.

Use Case: Add player Groups

Iteration: 1

Primary Actor: System

Goal in Context: Group matched players with their selected game

Preconditions: Players are grouped by skill based matchmaking system

Trigger: Player being grouped together

Scenario:

1. Matchmaking system has grouped players together based on skill.
2. Matchmaking system sends grouped players to match their game.
3. Groups are matched to their game worlds.

Post-Condition: Groups attached with their game world

Exceptions:

1. No game worlds are available for players to enter.
2. Network interrupt encountered.

Priority: High, required to play games

When Available: First development iteration

Frequency of Use: High

Channel to actor: Matchmaking system

Secondary Actor: Players

Channel to Secondary Actor: Game Screen

Open Issues:

1. Players cannot cancel matchmaking after reaching this level. They cannot quit the game midway.

Use Case: Signal Start Game

Iteration: 1

Primary Actor: System

Goal in Context: Signal the game system to start the game world simulation.

Preconditions: Player groups are matched with a game world

Trigger: Groups matched with a game world.

Scenario:

1. Player groups are matched with their game world.
2. The subsystem checks if game world is ready.
3. Signal sent to game system to start game world simulation.

Post-Condition: Game simulation starts

Exceptions:

1. Network interrupt causes game to be dropped.
2. The game system is operating at max capacity.

Priority: High, required to play games

When Available: First development iteration

Frequency of Use: High

Channel to actor: Matchmaking system

Secondary Actor: Players

Channel to Secondary Actor: Game Screen

Open Issues:

1. Player screen is not operational until the game simulation is loaded.

Use Case: Spectate Game

Iteration: 3

Primary Actor: System

Goal in Context: Signal database to join spectator player to join the game world

Preconditions: The spectator is not blacklisted and game world is running

Trigger: Spectator presses the spectate button

Scenario:

1. Spectator wants to view their friend's gameplay.
2. The system checks if the game world is active and spectator can join.
3. Player joins the game world simulation as spectator to their friend.

Post-Condition: Spectator can spectate the game in real time.

Exceptions:

1. Player is not authorized to spectate the game
2. Network issues disallow spectating.

Priority: Medium, required for increased player satisfaction.

When Available: third development iteration

Frequency of Use: Fair

Channel to actor: Matchmaking system

Secondary Actor: Players

Channel to Secondary Actor: Game Screen

Open Issues:

1. What happens when the game world is operating at max capacity and cannot host more players?

Use Case: View Leaderboard

Iteration: 1

Primary Actor: Player

Goal in Context: To allow the player to view the leaderboard and check rankings, scores, and statistics of other players.

Preconditions:

1. The player is logged into the system.
2. The leaderboard data is available and up to date.

Trigger: The player selects the "View Leaderboard" option from the game menu.

Scenario:

1. The player navigates to the leaderboard section.
2. The player can scroll through the leaderboard to view player rankings.
3. The player can search for a specific player's ranking.
4. The system updates the leaderboard in real-time based on new game results.

Postconditions:

1. The leaderboard information is displayed to the player.
2. Any new updates in player rankings are reflected in the displayed leaderboard.
3. The system logs the player's leaderboard viewing activity.

Exceptions:

1. The leaderboard data is unavailable due to a server issue.
2. The player's search for a specific ranking returns no results.

Priority: High

When Available: Available when the player selects the "Leaderboard" option from the game menu.

Frequency of Use: Often used, depending on player engagement and competition interest.

Channel to Actor: Game UI interface.

Secondary Actor: Database

Channel to Secondary Actor: Game Screen

Open Issues:

1. What happens when the player can't see the leaderboard for any or all games?

Use Case: View Stats**Iteration:** 1**Primary Actor:** layer

Goal in Context: To allow the player to view their game statistics, including win/loss records, match history, and performance trends.

Preconditions:

1. The player is logged into the system.
2. The system has recorded and stored the player's game statistics.

Trigger: The player selects the "View Stats" option from the game menu.

Scenario:

1. The player navigates to the stats section.
2. The system retrieves the player's game statistics.
3. The system displays the player's win/loss record, match history, and performance trends.
4. The player can filter or sort statistics based on game type, date, or opponent.
5. The system updates the statistics in real-time based on recent games.

Postconditions:

1. The player's game statistics are displayed.
2. Any new updates in player performance are reflected in the displayed statistics.
3. The system logs the player's stats viewing activity.

Exceptions:

1. The statistics data is unavailable due to a server issue.

2. The player's statistics have not been recorded yet (e.g., new account with no matches).

Priority: High

When Available: Available when the player selects the "Stats" option from the game menu.

Frequency of Use: Often used, depending on player engagement and interest in performance tracking.

Channel to Actor: Touchscreen display or game UI interface.

Secondary Actor: Database

Channel to Secondary Actor: Game Screen

Open Issues:

1. What happens if the player can't view his/her stats?

Use Case: Update Leaderboard

Iteration: 1

Primary Actor: Game System

Goal in Context: To update the leaderboard rankings based on the results of completed matches, ensuring accurate player rankings.

Preconditions:

1. A match has been completed.
2. The game system has recorded the match results.
3. The leaderboard is available and operational.

Trigger: A match concludes, and the system processes the results.

Scenario:

1. A match is completed.
2. The system retrieves the match results.
3. The system updates the player's ranking, win/loss record, and score.
4. The updated leaderboard data is stored in the system.
5. The leaderboard is refreshed and displayed with the new rankings.

Postconditions:

1. The leaderboard reflects the updated rankings.
2. The updated rankings are stored in the system database.
3. Players can see the latest leaderboard standings.

Exceptions:

1. A server error prevents the leaderboard from updating.
2. The match results cannot be retrieved due to data corruption.

Priority: High

When Available: Automatically triggered after a match is completed.

Frequency of Use: Frequently, depending on the number of matches played.

Channel to Actor: System backend process with data output to the leaderboard UI.

Secondary Actor: Database

Channel to Secondary Actor: Game Screen

Open Issues:

1. What happens if rank of the players doesn't get updated after each game?

Use Case: Game Result

Iteration: 1 Primary Actor: Game System

Goal in Context: To process and display the final result of a completed game, including winner, scores, and relevant statistics.

Preconditions:

1. A match has been completed.
2. The game system has recorded the match details and results.
3. The game session is still active to display the final result.

Trigger: The match reaches a conclusion (win/loss/draw).

Scenario:

1. The game detects the match has ended.
2. The system determines the winner, final scores, and relevant statistics.

3. The system updates the player's win/loss records.
4. The system displays the game result to all players.
5. The system stores the game result in the database for future reference.

Postconditions:

1. The final game result is displayed to the players.
2. The player's game records and statistics are updated.
3. The system logs the game result for future leaderboard updates and match history.

Exceptions:

1. A server error prevents the result from being recorded.
2. The game session crashes before results are displayed.

Priority: High

When Available: Automatically triggered at the end of a game session.

Frequency of Use: Every time a match is completed.

Channel to Actor: Game UI and system backend for processing and storage.

Secondary Actor: Database

Channel to Secondary Actor: Game Screen

Open Issues: N/A